Normalization Examples

Example I

Take the following table.

StudentID is the primary key.

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
					Maths	$50	A
					Info Tech	$100	B+

Is it 1NF?

No. There are repeating groups (subject, subjectcost, grade)

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English Maths Info Tech	$50 $50 $100	B A B+

How can you make it 1NF?

Create new rows so each cell contains only one value

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English Maths Info Tech	$50 $50 $100	B A B+

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

But now look – is the *studentID* primary key still valid?

No – the studentID no longer uniquely identifies each row

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

You now need to declare *StudentID* **and** *Subject* **together** to uniquely identify each row.

So the new **key** is StudentID *and* Subject.

So. We now have 1NF.

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

Is it 2NF?

(StudentID, Subject)

StudentID -> StudentName

StudentName and **Address** are dependent on studentID (which is part of the key)
This is good.

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

But they are **not** dependent on *Subject* (the *other* part of the key)

And 2NF requires…

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

All non-key fields are dependent on the ENTIRE key (StudentID + Subject)

So it's not 2NF

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

How can we fix it?

Make new tables

- Make a new table for each primary key field
- Give each new table its own primary key
- Move columns from the original table to the new table that matches their primary key

Step 1

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

STUDENT TABLE (key = StudentID)

Step 2

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Step 3

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

RESULTS TABLE (key = StudentID + Subject)

Step 3

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	$100	B+

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

RESULTS TABLE (key = StudentID + Subject)

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

Step 4 - relationships

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

Each student can only appear ONCE in the student table

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

Each subject can only appear ONCE in the subjects table

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

A subject can be listed MANY times in the results table (for different students)

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

1

A student can be listed MANY times in the results table (for different subjects)

∞

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 2NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞ ∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

SubjectCost is only dependent on the primary key, *Subject*

A 2NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

1

∞

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

Grade is only dependent on the primary key (*StudentID* + *Subject*)

A 2NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

Name, Address are only dependent on the primary key (*StudentID*)

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞ ∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

So it is 2NF!

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞ ∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

But is it 3NF?

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

Oh oh...

What?

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

HouseName is dependent on both ***StudentID + HouseColour***

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞ ∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

***Or* HouseColour** is dependent on both **StudentID + HouseName**

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞ ∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

But either way, non-key fields are dependent on MORE THAN THE PRIMARY KEY (StudentID)

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

*And 3NF says that non-key fields must depend on **nothing but the key***

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞ ∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

WHAT DO WE DO?

SUBJECTS TABLE (key = Subject)

1

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

∞

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

Again, carve off the offending fields

StudentTable

StudentID	StudentName	Address	HouseName
19594332X	Mary Watson	10 Charles Street	Bob

Primary key: StudentID

1

∞

∞

1

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

A 3NF fix

StudentTable

StudentID	StudentName	Address
19594332X	Mary Watson	10 Charles Street

Primary key: StudentID

HouseTable

HouseName	HouseColor
Bob	Red

Primary key: HouseName

1

∞

∞

1

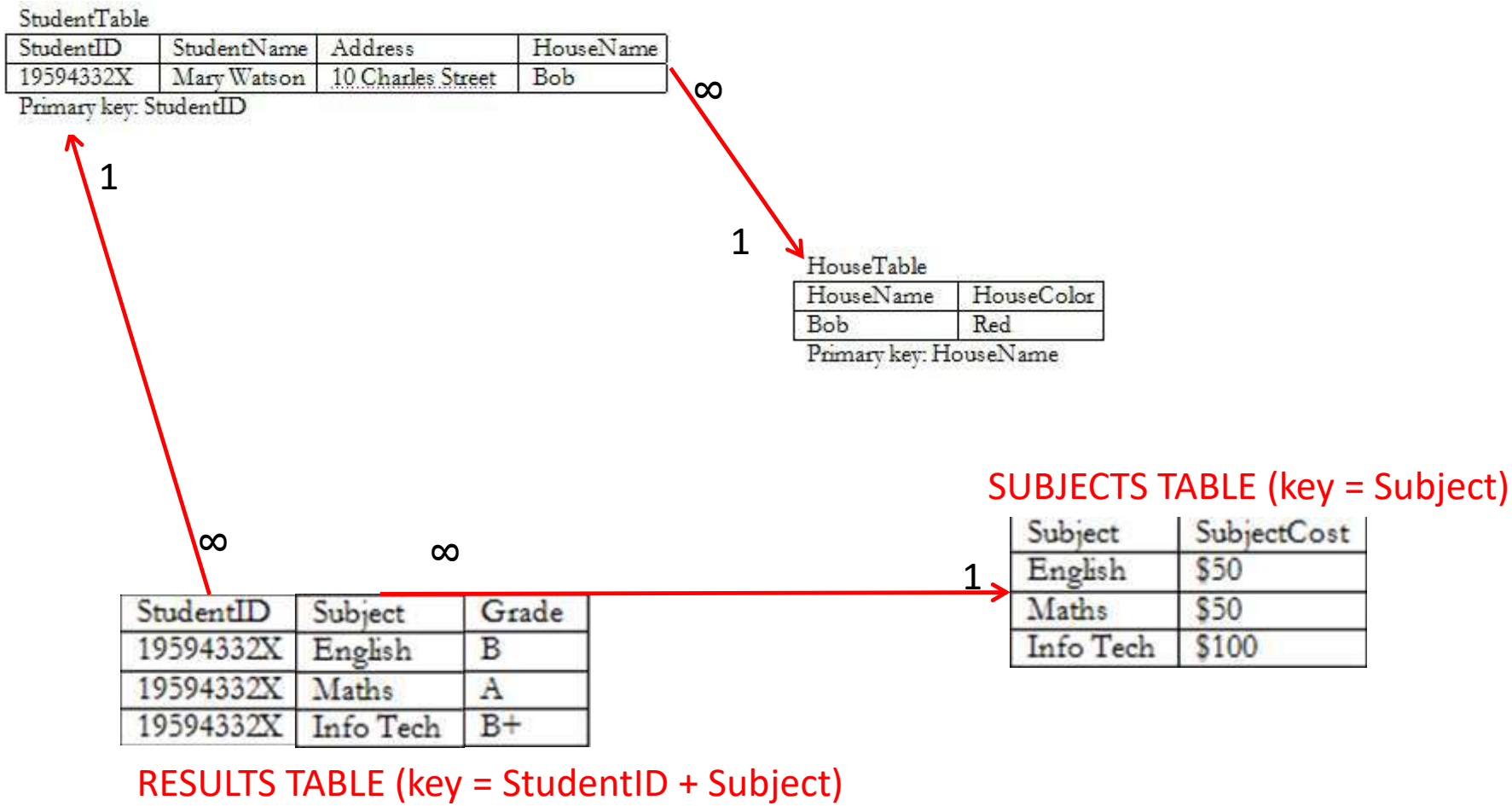
SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

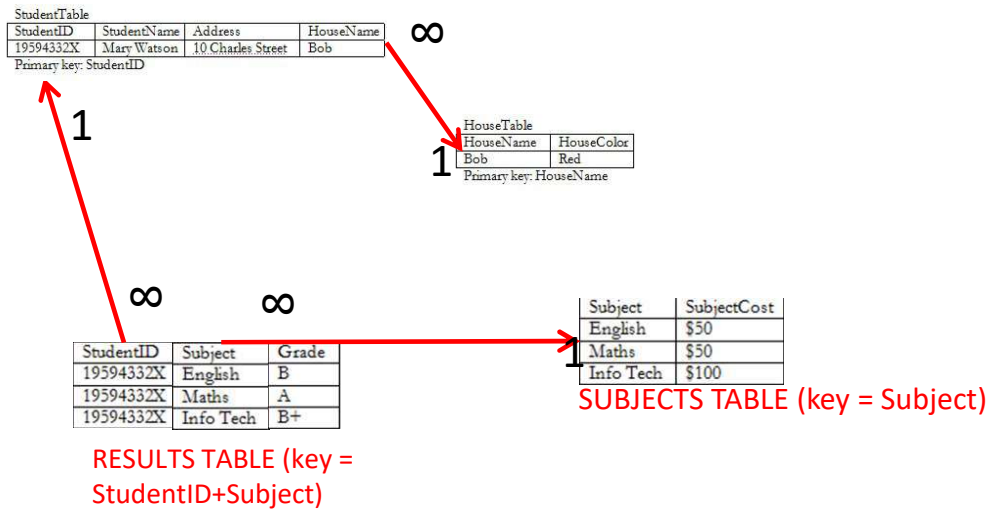
StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID + Subject)

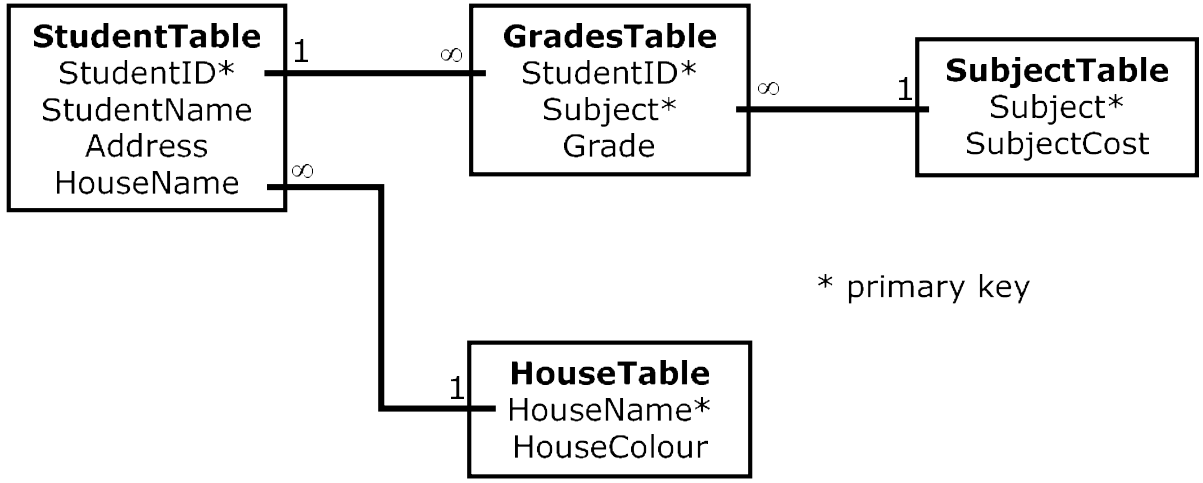
A 3NF fix


StudentTable
StudentID | StudentName | Address | HouseName
19594332X | Mary Watson | 10 Charles Street | Bob
Primary key: StudentID
∞
1
1
HouseTable
HouseName | HouseColor
Bob | Red
Primary key: HouseName
SUBJECTS TABLE (key = Subject)
Subject | SubjectCost
English | $50
Maths | $50
Info Tech | $100
∞
∞
1
StudentID | Subject | Grade
19594332X | English | B
19594332X | Maths | A
19594332X | Info Tech | B+
RESULTS TABLE (key = StudentID + Subject)

A 3NF win!


StudentTable
StudentID | StudentName | Address | HouseName
19594332X | Mary Watson | 10 Charles Street | Bob
Primary key: StudentID
∞
1
HouseTable
HouseName | HouseColor
Bob | Red
Primary key: HouseName
1
∞
∞
StudentID | Subject | Grade
19594332X | English | B
19594332X | Maths | A
19594332X | Info Tech | B+
Subject | SubjectCost
English | $50
Maths | $50
Info Tech | $100
1
SUBJECTS TABLE (key = Subject)
RESULTS TABLE (key = StudentID+Subject)

Or…


StudentTable
StudentID*
StudentName
Address
HouseName
1
∞
∞
GradesTable
StudentID*
Subject*
Grade
∞
1
SubjectTable
Subject*
SubjectCost
1
HouseTable
HouseName*
HouseColour
* primary key

The Reveal

Before…

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	$50	B
					Maths	$50	A
					Info Tech	$100	B+

After…

HouseTable

HouseName	HouseColor
Bob	Red

Primary key: HouseName

StudentTable

StudentID	StudentName	Address	HouseName
19594332X	Mary Watson	10 Charles Street	Bob

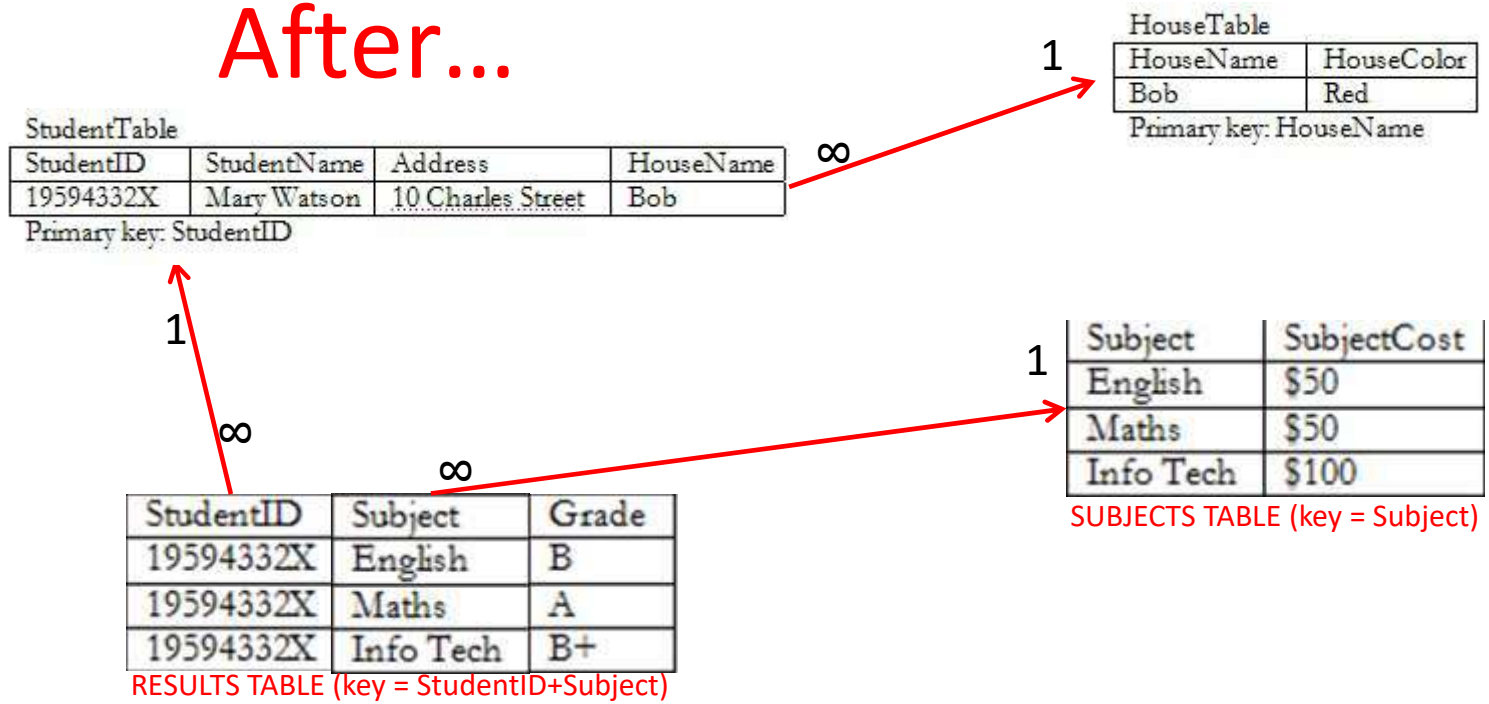
Primary key: StudentID

Subject	SubjectCost
English	$50
Maths	$50
Info Tech	$100

SUBJECTS TABLE (key = Subject)

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



Example II

Normalization Example

Orders(<u>Order</u>, <u>Product</u>, Quantity, UnitPrice, Customer, Address)

- We have a table representing orders in an online store
- Each row represents an item on a particular order
- Primary key is {Order, Product}

Functional Dependencies

Orders(<u>Order</u>, <u>Product</u>, Quantity, UnitPrice, Customer, Address)

- Each order is for a single customer:
  - Order $\rightarrow$ Customer
- Each customer has a single address
  - Customer $\rightarrow$ Address
- Each product has a single price
  - Product $\rightarrow$ UnitPrice
- As Order $\rightarrow$ Customer and Customer $\rightarrow$ Address
  - Order $\rightarrow$ Address

Order -> Customer, Address

2NF Solution (I)

- ***First decomposition***
  - First table

<u>Order</u>	<u>Product</u>	Quantity	UnitPrice

  - Second table

<u>Order</u>	Customer	Address

2NF Solution (II)

- ***Second decomposition***
  - First table

<u>Order</u>	<u>Product</u>	Quantity

  - Second table

<u>Order</u>	Customer	Address

  - Third table

<u>Product</u>	UnitPrice

3NF

- In second table

<u>Order</u>	Customer	Address

  – Customer $\rightarrow$ Address

- Split second table into

<u>Order</u>	Customer

<u>Customer</u>	Address

Normalization to 2NF

- Second normal form means no partial dependencies on candidate keys
  - {Order} $\rightarrow$ {Customer, Address}
  - {Product} $\rightarrow$ {UnitPrice}

- To remove the first FD we project over

  {Order, Customer, Address} (R1)

and

  {Order, Product, Quantity, UnitPrice} (R2)

Normalization to 2NF

- R1 is now in 2NF, but there is still a partial FD in R2

  {Product} $\rightarrow$ {UnitPrice}

- To remove this we project over

  {Product, UnitPrice} (R3)

  and

  {Order, Product, Quantity} (R4)

Normalization to 3NF

- R has now been split into 3 relations - R1, R3, and R4
  - R3 and R4 are in 3NF
  - R1 has a transitive FD on its key
- To remove

  $\{Order\} \rightarrow \{Customer\} \rightarrow \{Address\}$
- we project R1 over
  - {Order, Customer}
  - {Customer, Address}

Normalization

- 1NF:
  - {<u>Order, Product</u>, Customer, Address, Quantity, UnitPrice}
- 2NF:
  - {<u>Order</u>, Customer, Address}, {<u>Product</u>, UnitPrice}, and {<u>Order, Product</u>, Quantity}
- 3NF:
  - {<u>Product</u>, UnitPrice}, {<u>Order, Product</u>, Quantity}, {<u>Order</u>, Customer}, and {<u>Customer</u>, Address}

DATABASE DESIGN: NORMALIZATION NOTE & EXERCISES (Up to 3NF)

- Tables that contain redundant data can suffer from update anomalies, which can introduce inconsistencies into a database.
- The rules associated with the most commonly used normal forms, namely first (1NF), second (2NF), and third (3NF).
- The identification of various types of update anomalies such as insertion, deletion, and modification anomalies can be found when tables that break the rules of 1NF, 2NF, and 3NF and they are likely to contain redundant data and suffer from update anomalies.
- Normalization is a technique for producing a set of tables with desirable properties that support the requirements of a user or company.
- Major aim of relational database design is to group columns into tables to minimize data redundancy and reduce file storage space required by base tables.
- Take a look at the following example:

StdSSN	StdCity	StdClass	OfferNo	OffTerm	OffYear	EnrGrade	CourseNo	CrsDesc
S1	SEATTLE	JUN	O1	FALL	2006	3.5	C1	DB
S1	SEATTLE	JUN	O2	FALL	2006	3.3	C2	VB
S2	BOTHELL	JUN	O3	SPRING	2007	3.1	C3	OO
S2	BOTHELL	JUN	O2	FALL	2006	3.4	C2	VB

- The **insertion anomaly**: Occurs when extra data beyond the desired data must be added to the database. For example, to insert a course (`CourseNo`), it is necessary to know a student (`StdSSN`) and offering (`OfferNo`) because the combination of `StdSSN` and `OfferNo` is the primary key. Remember that a row cannot exist with `NULL` values for part of its primary key.
- The **update anomaly**: Occurs when it is necessary to change multiple rows to modify **ONLY** a single fact. For example, if we change the `StdClass` of student `S1` (`JUN`), two rows, row 1 and 2 must be changed. If `S1` was enrolled in 10 classes, 10 rows must be changed.
- The **deletion anomaly**: Occurs whenever deleting a row inadvertently causes other data to be deleted. For example, if we delete the enrollment (`EnrGrade`) of `S2` in `O3` (third row), we lose the information about offering `O3` and course `C3` because these values are unique to the table (cell). Furthermore `O3` is a primary key.

RECAP

Problems associated with data redundancy are illustrated by comparing the `Staff` and `Branch` tables with the `StaffBranch` table. Tables that have redundant data may have problems called update anomalies, which are classified as insertion, deletion, or modification anomalies. See the following Figure for an example of a table with redundant data called `StaffBranch`. There are two main types of insertion anomalies, which we illustrate using this table.

Insertion anomalies

1. To insert the details of a new member of staff (staffNo, name, position and salary) located at a given branch into the StaffBranch table, we must also enter the correct details for that branch (branchNo, branchAddress and telNo). For example, to insert the details of a new member of staff at branch B002, we must enter the correct details of branch B002 so that the branch details are consistent with values for branch B002 in other records of the StaffBranch table. The data shown in the StaffBranch table is also shown in the Staff and Branch tables. These tables do have redundant data and do not suffer from this potential inconsistency, because for each staff member we only enter the appropriate branch number into the Staff table. In addition, the details of branch B002 are recorded only once in the database as a single record in the Branch table.

2. To insert details of a new branch that currently has no members of staff into the StaffBranch table, it's necessary to enter NULLs into the staff-related columns, such as staffNo. However, as staffNo is the primary key for the StaffBranch table, attempting to enter nulls for staffNo violates entity integrity, and is not allowed. The design of the tables shown in Staff and Branch avoids this problem because new branch details are entered into the Branch table separately from the staff details. The details of staff ultimately located at a new branch can be entered into the Staff table at a later date.

Deletion anomalies

If we delete a record from the StaffBranch table that represents the last member of staff located at a branch, the details about that branch are also lost from the database. For example, if we delete the record for staff Art Peters (S0415) from the StaffBranch table, the details relating to branch B003 are lost from the database. The design of the tables that separate the Staff and Branch table avoids this problem because branch records are stored separately from staff records and only the column branchNo relates the two tables. If we delete the record for staff Art Peters (S0415) from the Staff table, the details on branch B003 in the Branch table remain unaffected.

Modification anomalies

If we want to change the value of one of the columns of a particular branch in the StaffBranch table, for example the telephone number for branch B001, we must update the records of all staff located at that branch (row 1 and 2). If this modification is not carried out on all the appropriate records of the StaffBranch table, the database will become inconsistent. In this example, branch B001 would have different telephone numbers in different staff records.
The above examples illustrate that the Staff and Branch tables have more desirable properties than the StaffBranch table.

StaffBranch Table

staffNo	name	position	salary	branchNo	branchAddress	telNo
S1500	Tom Daniels	Manager	46000	B001	8 Jefferson Way, Portland, OR 97201	503-555-3618
S0003	Sally Adams	Assistant	30000	B001	8 Jefferson Way, Portland, OR 97201	503-555-3618
S0010	Mary Martinez	Manager	50000	B002	City Center Plaza, Seattle, WA 98122	206-555-6756
S3250	Robert Chin	Supervisor	32000	B002	City Center Plaza, Seattle, WA 98122	206-555-6756
S2250	Sally Stern	Manager	48000	B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131
S0415	Art Peters	Manager	41000	B003	14 – 8th Avenue, New York, NY 10012	212-371-3000

Staff and Branch Tables

Staff

staffNo	name	position	salary	branchNo
S1500	Tom Daniels	Manager	46000	B001
S0003	Sally Adams	Assistant	30000	B001
S0010	Mary Martinez	Manager	50000	B002
S3250	Robert Chin	Supervisor	32000	B002
S2250	Sally Stern	Manager	48000	B004
S0415	Art Peters	Manager	41000	B003

Branch

branchNo	branchAddress	telNo
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618
B002	City Center Plaza, Seattle, WA 98122	206-555-6756
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131

- `StaffBranch` table has redundant data; **the details of a branch** are repeated for every member of staff for example row 1 and 2, row 3 and 4 on `branchNo`, `branchAddress` and `telNo`.
- In contrast, the branch information appears only once for each branch in the `Branch` table and only the branch number (`branchNo`) is repeated in the `Staff` table, to represent where each member of staff is located.
- Let take a look at another example.

AIRCRAFT_1 Table

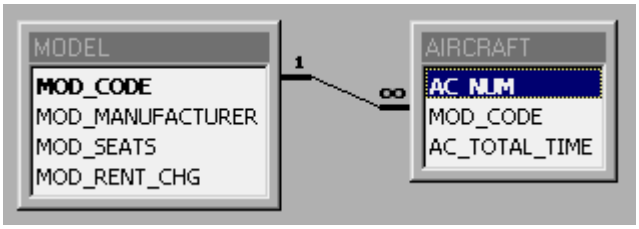
AC_NUM	AC_MODEL	AC_RENT_CHG	AC_SEATS	AC_TOTAL_TIME
1178R	Cessna C-172 Shyhawk	$58.50	4	4512.6
205IY	Cessna C-172 Shyhawk	$58.50	4	5325.5
2087V	Cessna C-152 Commuter	$51.75	2	4889.8
226BR	Cessna C-172 Shyhawk	$58.50	4	4299.6
2867W	Piper PA28-181 Archer II	$64.00	4	3267.4
3213R	Piper PA28-181 Archer II	$64.00	4	2517.9
4112E	Piper PA28-181 Archer II	$64.00	4	5211.3
45ZU	Cessna C-152 Commuter	$51.75	2	7003.1
5725Y	Cessna C-172 Shyhawk	$58.50	4	3968.2

- If we use the `AIRCRAFT_1` table as shown in the above Figure, a change in hourly rental rates (`AC_RENT_CHG`) for the `Cessna 172 Skyhawk` must be made four times; if we forget to change just one of those rates, we have a data integrity problem.
- How much better it would be to have critical data in only one place. Then, if a change must be made, it need be made only once.
- In contrast, table structures are good when they preclude the possibility of producing uncontrolled data redundancies.
- We can produce such a happy circumstance by splitting the `AIRCRAFT_1` table as shown in the following two Figures, connecting the two resulting tables through the `AIRCRAFT_1` table's foreign key `MOD_CODE`.

AC_NUM	MOD_CODE	AC_TOTAL_TIME
1178R	C-172	4512.6
205IY	C-172	5325.5
2087V	C-152	4889.8
226BR	C-172	4299.6
2867W	PA28-181	3267.4
3213R	PA28-181	2517.9
4112E	PA28-181	5211.3
45ZU	C-152	7003.1
5725Y	C-172	3968.2

MOD_CODE	MOD_MANUFACTURER	MOD_SEATS	MOD_RENT_CHG
C-152	Cessna	2	$51.75
C-172	Cessna	4	$58.50
PA28-181	Piper	4	$64.00

- Note that a rental rate change need be made in only one place, a description is given in only one place, and so on. No more data update and delete anomalies and no more data integrity problems. The relational schema in the following Figure shows how the two tables are related.



The First normal form (1NF)

- A table in which the intersection of every column and record contains only one value. It prohibits nesting or repeating groups in table. The intersection must be atomic.
- For example the `telNos` column contains multiple values.

branchNo	branchAddress	telNos
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618, 503-555-2727, 503-555-6534
B002	City Center Plaza, Seattle, WA 98122	206-555-6756, 206-555-8836
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131, 206-555-4112

Primary key

More than one value, so *not* in 1NF

Branch (Not 1NF)

branchNo	branchAddress	telNos
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618, 503-555-2727, 503-555-6534
B002	City Center Plaza, Seattle, WA 98122	206-555-6756, 206-555-8836
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131, 206-555-4112

Take copy of branchNo column to new table to become foreign key

Remove telNos column and create new column called telNo in the new table

Branch (1NF)

branchNo	branchAddress
B001	8 Jefferson Way, Portland, OR 97201
B002	City Center Plaza, Seattle, WA 98122
B003	14 – 8th Avenue, New York, NY 10012
B004	16 – 14th Avenue, Seattle, WA 98128

Primary key

BranchTelephone (1NF)

branchNo	telNo
B001	503-555-3618
B001	503-555-2727
B001	503-555-6534
B002	206-555-6756
B002	206-555-8836
B003	212-371-3000
B004	206-555-3131
B004	206-555-4112

Becomes foreign key

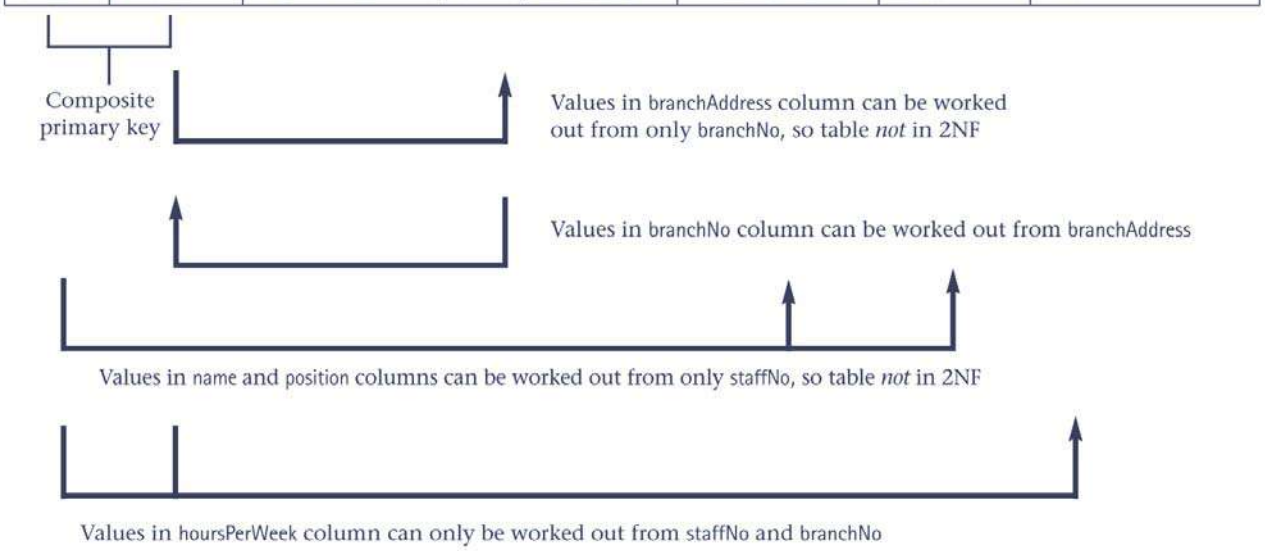
Becomes primary key

The Second normal form (2NF)

- 2NF ONLY applies to tables with **composite primary keys** (more than one primary key).
- A table that is in 1NF and in which the values of each **non-primary-key column** can be worked out from the values in ALL the columns that make up the primary key.

- A table is in 2NF if **each non-key (non primary and/or candidate keys) column depends on ALL candidate keys**, **NOT on a subset of ANY candidate key**.
- The 2NF violation occurs when Functional Dependency (FD) in which part of key (instead of the whole keys) determines a non-key. An FD containing a single column Left Hand Side (LHS) cannot violate 2NF.
- For example, `TempStaffAllocation` table in the following Figure is in 2NF because `branchAddress` can depend on `branchNo` only not both of `staffNo` AND `branchNo` (`staffNo` & `branchNo` are candidate keys and at the same time can be primary keys and at the same time is composite key because more than one primary keys). Another one is the values in `name` and `position` columns can depend on (can stand on it own) the `staffNo` ONLY not both of `staffNo` and `branchNo`. What we want is the `hoursperWeek` column, which depends on both `staffNo` and `branchNo`. In another word we must avoid the partial dependencies on the candidate keys.

staffNo	branchNo	branchAddress	name	position	hoursPerWeek
S4555	B002	City Center Plaza, Seattle, WA 98122	Ellen Layman	Assistant	16
S4555	B004	16 – 14th Avenue, Seattle, WA 98128	Ellen Layman	Assistant	9
S4612	B002	City Center Plaza, Seattle, WA 98122	Dave Sinclair	Assistant	14
S4612	B004	16 – 14th Avenue, Seattle, WA 98128	Dave Sinclair	Assistant	10



The Functional dependency

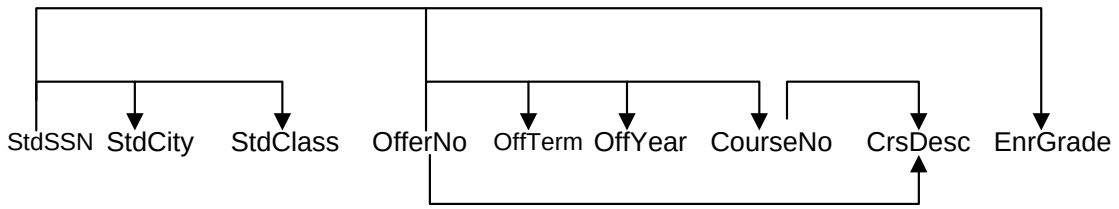
- The particular **relationships that we show between the columns of a table** are more formally referred to as **functional dependencies** (FDs). FD describes the relationship between columns in a table.
- Another example, consider a table with columns A and B, where B is functionally dependent on A (denoted $A \rightarrow B$). If we know the value of A, we find only one value of B in all the records that has this value of A, at any moment in time.

FD Definition

- $X \rightarrow Y$.

- X (functionally) determines Y or Y is functionally dependent on X.
- X: left-hand-side (LHS) or determinant.
- For each X value, there is at most one Y value.
- Similar to candidate keys.

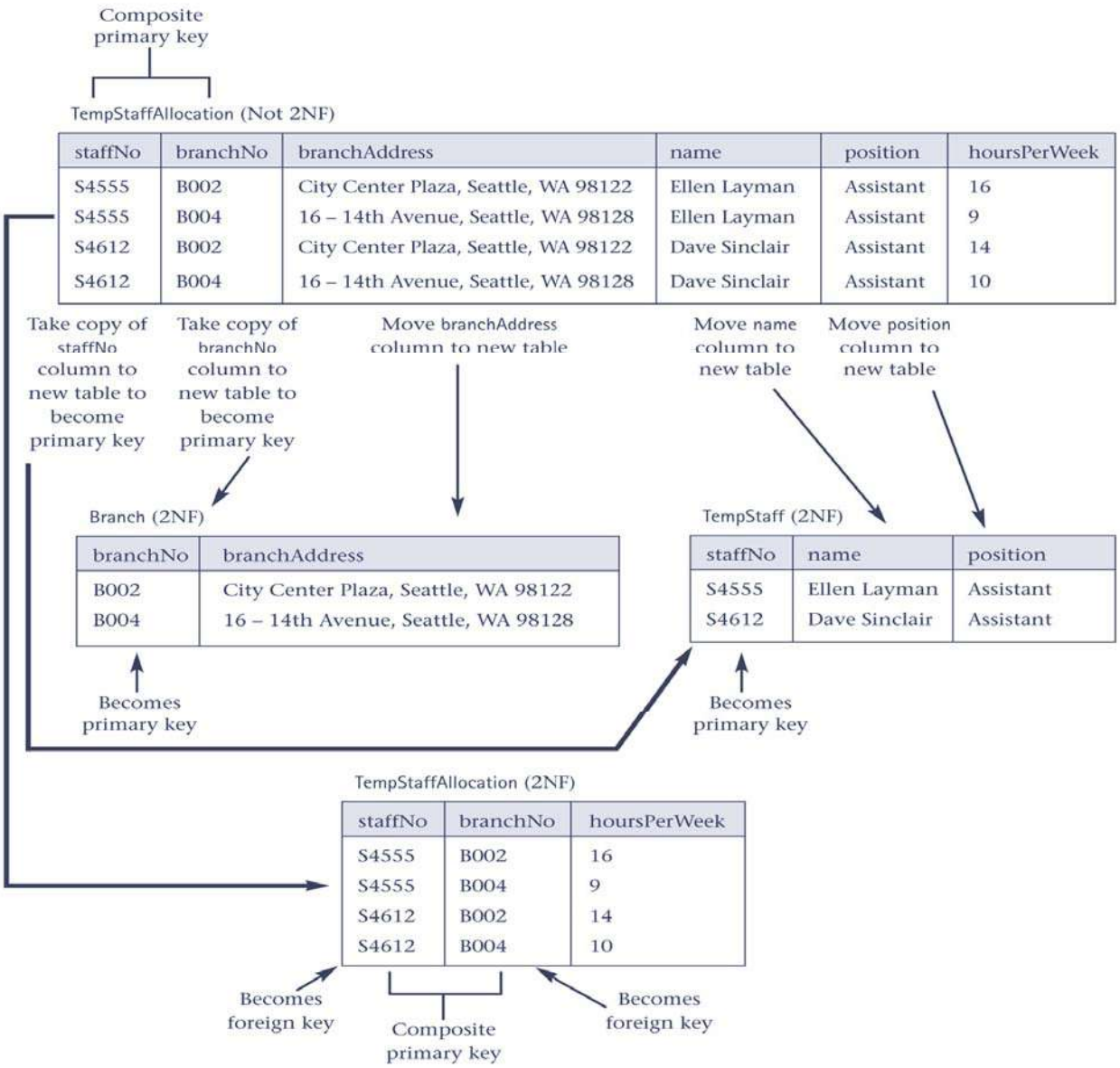
- For example (take note regarding the arrow flow!):



- The FDs are (another notation used to write FDs):

```
StdSSN → StdCity, StdClass
OfferNo → OffTerm, OffYear, CourseNo, CrsDesc
CourseNo → CrsDesc
StdSSN, OfferNo → EnrGrade
```

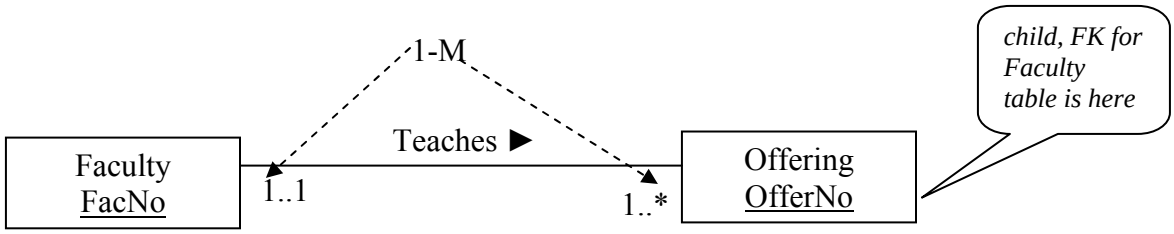
- Formal definition of 2NF is a table that is in 1NF and every non-primary-key column is **fully functional dependent** on the primary key.
- **Full functional dependency** indicates that if A and B are columns of a table, B is fully dependent on A if B is functionally dependent on A but not on any proper subset of A.
- Consider the following examples.



Identifying Functional Dependencies

- Database designers must be able to identify FD when collecting database requirements.
- In problem narratives, some FD can be identified by statements about uniqueness. For example a user may state that each course offering has a unique offering number along with the year and term of the offering. From this statement, the designer should assert that `OfferNo` → `OffYear` and `OffTerm`.
- You can also identify functional dependencies in a table design resulting from the conversion of an ERD. FD would be asserted for each unique column (PK or other candidate key) with the unique column as the LHS and other columns in the table on the RHS.
- Although FD derived from statements about 1-M relationships can be identify, FD derived from statements about 1-M relationship can be confusing to identify.
- When you see a statement about a 1-M relationship, the FD is derived from the child-to-parent direction, not the parent-to-child direction.

- For example, the statement "A faculty teaches many offerings but an offering is taught by one faculty" defines a FD from a unique column of offering to a unique column of faculty such as `OfferNo` → `FacNo`.
- Novice designers sometimes incorrectly assert that `FacNo` determines a collection of `OfferNo` values. This statement is not correct because a **FD must allow at most one associated value**, not a collection of values.



OfferNo → FacNo **NOT** FacNo → OfferNo

- FD in which the LHS is not a PK or candidate key can also be difficult to identify.
- These FDs are especially important to identify after converting an ERD to a table design. You should carefully look for FDs in which the LHS is not a candidate key or primary key.
- You should also consider FDs in tables with a combined PK or candidate key in which the LHS is part of a key, but not the entire key. The NFs these kinds of FDs can lead to modification anomalies.
- Another important consideration in asserting FD is the minimalism of LHS. It is important to distinguish when one column alone is the determinant versus a combination of column.
- An FD in which the LHS contains more than one column usually represents an M-N relationship. For example, the statement "The order quantity is collected for each product purchased in an order" translates to the FD `OrdNo`, `ProdNo` → `OrdQty`.
- Order quantity depends on the combination of order number and product number, not just one of these columns.
- Part of the confusion about the minimalism of the LHS is due to the meaning of columns in the left-hand versus right-hand side of a dependency. The minimal determinant means the determinant (column(s) appearing on the LHS of FD) must not contain extra columns and this minimalism requirement is similar to the minimalism requirement for candidate keys.
- For example, to record that student Social Security Number determines city and class, you can write either:

  `StdSSN` → `StdCity, StdClass` (more compact) or

  `StdSSN` → `StdCity` and `StdSSN` → `StdClass` (less compact)

- If you assume that the email address is also unique for each student, then you can write:

  `Email` → `StdCity, StdClass`

- You should not write:

  `StdSSN, Email → StdCity, StdClass`

- This is because these FDs imply that the combination of `StdSSN` and `Email` is the determinant. Thus you should write FDs so that the LHS does not contain unneeded columns.
- The prohibition against unneeded columns for determinants is the same as the prohibition against unneeded columns in candidate keys. Both determinants and candidate keys must be minimal.
- A FD cannot be proven to exist by examining the rows of a table. However you can falsify a FD (i.e. prove that a FD does not exist) by examining the contents of a table.
- For example, in the university database, we can conclude that `StdClass` does not determine `StdCity` because there are two rows with the same value for `StdClass` but a different value for `StdCity`.
- Thus, it is sometimes helpful to examine sample rows in a table to eliminate potential functional dependencies.
- There are several commercial database design tools that automate the process of eliminating dependencies through examination of sample rows. Ultimately, the database designer must make the final decision about FDs that exist in a table.

Third normal form (3NF)

- A table that is in 1NF and 2NF and in which **all non-primary-key column can be worked out from only the primary key column(s) and no other columns**.
- At this level, the combined definition of 2NF and 3NF is a table is in 3NF if each non-key column depends on all candidate keys, whole candidate keys and nothing but candidate keys.
- For 2NF we should remove **partial dependency** and for 3NF we should remove **transitive dependency**.
- For example the `StaffBranch` table is not in 3NF.

StaffBranch (Not 3NF)

staffNo	name	position	salary	branchNo	branchAddress	telNo
S1500	Tom Daniels	Manager	46000	B001	8 Jefferson Way, Portland, OR 97201	503-555-3618
S0003	Sally Adams	Assistant	30000	B001	8 Jefferson Way, Portland, OR 97201	503-555-3618
S0010	Mary Martinez	Manager	50000	B002	City Center Plaza, Seattle, WA 98122	206-555-6756
S3250	Robert Chin	Supervisor	32000	B002	City Center Plaza, Seattle, WA 98122	206-555-6756
S2250	Sally Stern	Manager	48000	B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131
S0415	Art Peters	Manager	41000	B003	14 – 8th Avenue, New York, NY 10012	212-371-3000

Primary key

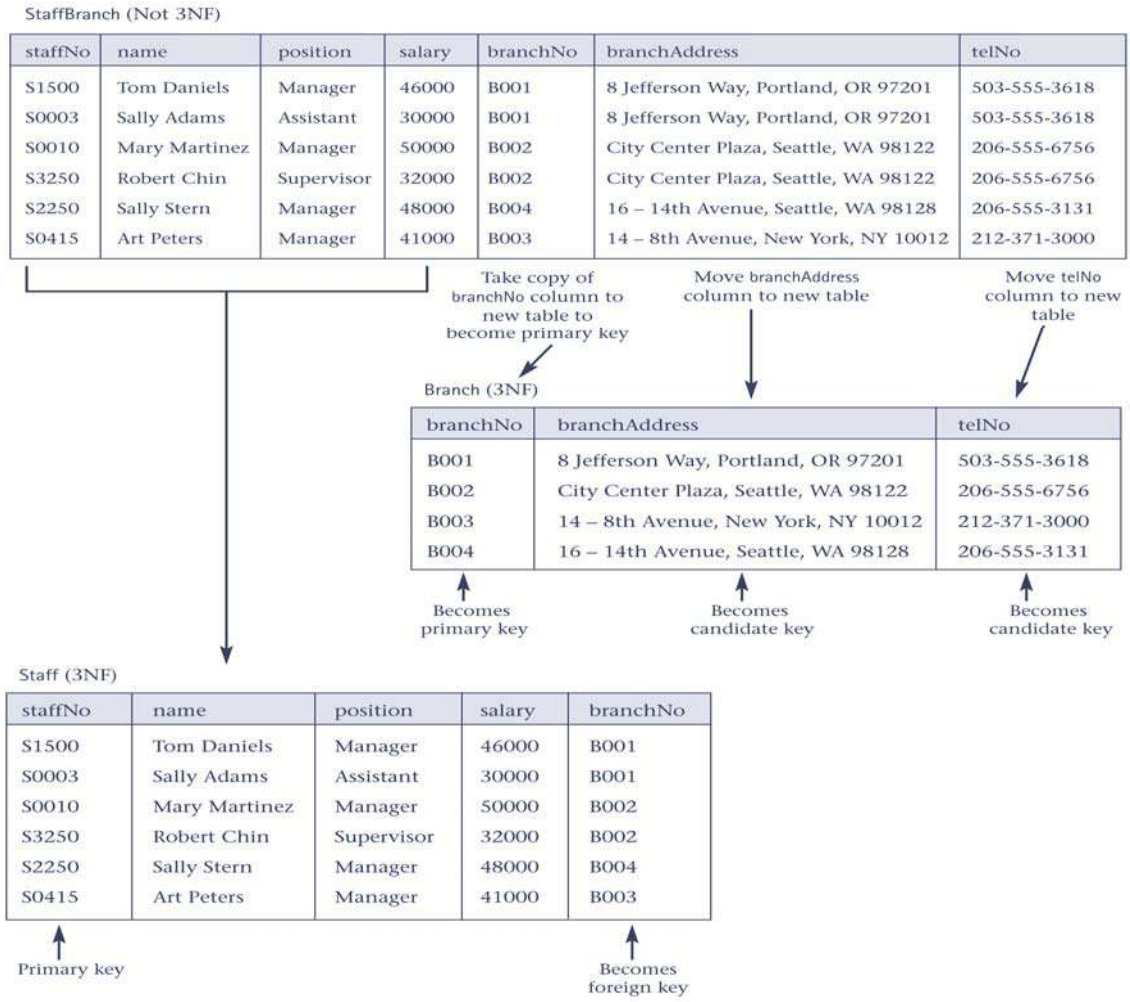
Values in all non-primary-key columns can be worked out from the primary key, staffNo

Values in branchAddress and telNo columns can be worked out from branchNo, so table *not* in 3NF

Values in branchNo and telNo columns can be worked out from branchAddress, so table *not* in 3NF

Values in branchNo and branchAddress columns can be worked out from telNo, so table *not* in 3NF

- The formal definition of 3NF is a table that is in 1NF and 2NF and in which no non-primary-key column is transitively dependent on the primary key.
- For example, consider a table with A, B, and C. If B is functional dependent on A ($A \rightarrow B$) and C is functional dependent on B ($B \rightarrow C$), then C is **transitively dependent** on A via B (provided that A is not functionally dependent on B or C).
- If a transitive dependency exists on the primary key, the table is not in 3NF.



Reviews Questions & Answers

1. Describe the characteristics of a table that violates first normal form (1NF) and then describe how such a table is converted to 1NF.

   The rule for first normal form (1NF) is a table in which the intersection of every column and record contains only one value. In other words a table that contains more than one atomic value in the intersection of one or more column for one or more records is not in 1NF. The non 1NF table can be converted to 1NF by restructuring original table by removing the column with the multi-values along with a copy of the primary key to create a new table. The advantage of this approach is that the resultant tables may be in normal forms later that 1NF.

2. What is the minimal normal form that a relation must satisfy? Provide a definition for this normal form.

   Only first normal form (1NF) is critical in creating appropriate tables for relational databases. All the subsequent normal forms are optional. However, to avoid the update anomalies, it's normally recommended that you proceed to third normal form (3NF).

First normal form (1NF) is a table in which the intersection of every column and record contains only one value.

3. Describe an approach to converting a first normal form (1NF) table to second normal form (2NF) table(s).

   Second normal form applies only to tables with composite primary keys, that is, tables with a primary key composed of two or more columns. A 1NF table with a single column primary key is automatically in at least 2NF.
   A second normal form (2NF) is a table that is already in 1NF and in which the values in each non-primary-key column can be worked out from the values in all the columns that makes up the primary key.
   A table in 1NF can be converted into 2NF by removing the columns that can be worked out from only part of the primary key. These columns are placed in a new table along with a copy of the part of the primary key that they can be worked out from.

4. Describe the characteristics of a table in second normal form (2NF).

   Second normal form (2NF) is a table that is already in 1NF and in which the values in each non-primary-key column can only be worked out from the values in all the columns that make up the primary key.

5. Describe what is meant by full functional dependency and describe how this type of dependency relates to 2NF. Provide an example to illustrate your answer.

   The formal definition of second normal form (2NF) is a table that is in first normal form and every non-primary-key column is fully functionally dependent on the primary key. Full functional dependency indicates that if A and B are columns of a table, B is fully functionally dependent on A, if B is not dependent on any subset of A. If B is dependent on a subset of A, this is referred to as a partial dependency. If a partial dependency exists on the primary key, the table is not in 2NF. The partial dependency must be removed for a table to achieve 2NF.

6. Describe the characteristics of a table in third normal form (3NF).

   Third normal form (3NF) is a table that is already in 1NF and 2NF, and in which the values in all non-primary-key columns can be worked out from only the primary key (or candidate key) column(s) and no other columns.

7. Describe what is meant by transitive dependency and describe how this type of dependency relates to 3NF. Provide an example to illustrate your answer.

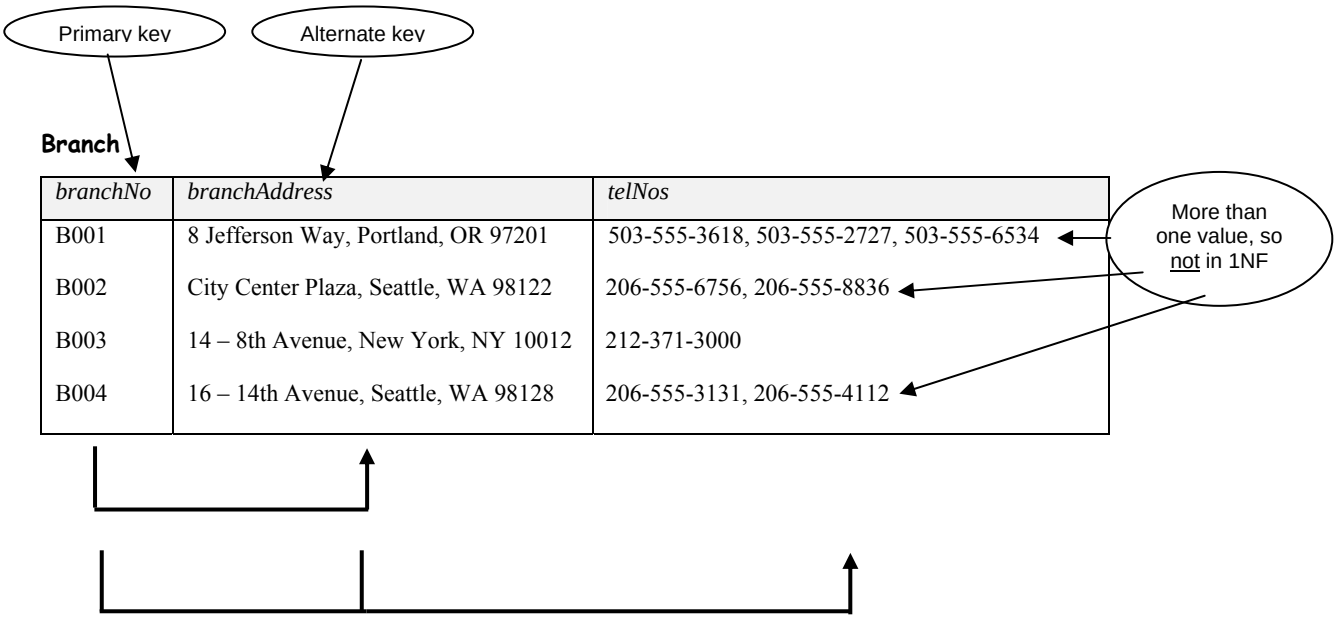
   The formal definition for third normal form (3NF) is a table that is in first and second normal forms and in which no non-primary-key column is transitively dependent on the primary key. Transitive dependency is a type of functional dependency that occurs when a particular type of relationship holds between columns of a table. For example, consider a table with columns A, B, and C. If B is functionally dependent on A ($A \rightarrow B$) and C is functionally dependent on B ($B \rightarrow C$), then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C). If a transitive dependency exists on the primary key, the table is not in 3NF. The transitive dependency must be removed for a table to achieve 3NF.

8. Examine the table shown below.

branchNo	*branchAddress*	*telNos*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618, 503-555-2727, 503-555-6534
B002	City Center Plaza, Seattle, WA 98122	206-555-6756, 206-555-8836
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131, 206-555-4112

(a) Why is this table not in 1NF?
(b) Describe and illustrate the process of normalizing the data shown in this table to third normal form (3NF).
(c) Identify the primary, alternate and foreign keys in your 3NF relations.

Answer:



branchNo	*branchAddress*	*telNos*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618, 503-555-2727, 503-555-6534
B002	City Center Plaza, Seattle, WA 98122	206-555-6756, 206-555-8836
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131, 206-555-4112

Branch

branchNo	*branchAddress*	*telNos*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618, 503-555-2727, 503-555-6534
B002	City Center Plaza, Seattle, WA 98122	206-555-6756, 206-555-8836
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131, 206-555-4112

Take copy of branchNo

Remove telNos column and create new column called telNo

Branch

branchNo	*branchAddress*
B001	8 Jefferson Way, Portland, OR 97201
B002	City Center Plaza, Seattle, WA 98122
B003	14 – 8th Avenue, New York, NY 10012
B004	16 – 14th Avenue, Seattle, WA 98128

Primary key

Alternate kev

BranchTelephone

branchNo	*telNo*
B001	503-555-3618
B001	503-555-2727
B001	206-555-6534
B002	206-555-6756
B002	206-555-8836
B003	212-371-3000
B004	206-555-3131
B004	206-555-4112

Becomes foreign key

Becomes primary key

(Note: There is an alternative approach to altering the original `Branch` table – columns can be added to the original `Branch` table to hold the individual values for each telephone number, e.g. `telNo1`, `telNo2` and `telNo3`.)

9. Examine the table shown below.

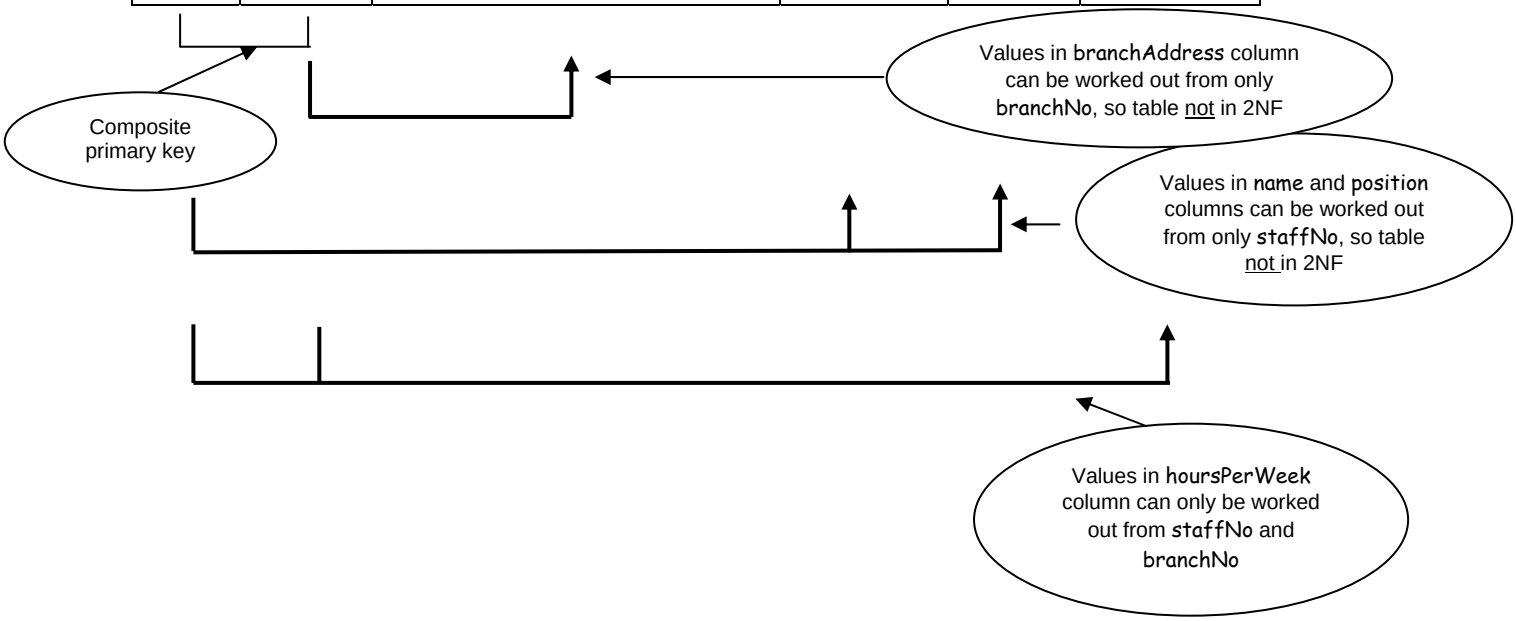
staffNo	*branchNo*	*branchAddress*	*name*	*position*	*hoursPerWeek*
S4555	B002	City Center Plaza, Seattle, WA 98122	Ellen Layman	Assistant	16
S4555	B004	16 – 14th Avenue, Seattle, WA 98128	Ellen Layman	Assistant	9
S4612	B002	City Center Plaza, Seattle, WA 98122	Dave Sinclair	Assistant	14
S4612	B004	16 – 14th Avenue, Seattle, WA 98128	Dave Sinclair	Assistant	10

(a) Why is this table not in 2NF?
(b) Describe and illustrate the process of normalizing the data shown in this table to third normal form (3NF).
(c) Identify the primary, (alternate) and foreign keys in your 3NF relations.

Answer:

TempStaffAllocation

staffNo	*branchNo*	*branchAddress*	*name*	*position*	*hoursPerWeek*
S4555	B002	City Center Plaza, Seattle, WA 98122	Ellen Layman	Assistant	16
S4555	B004	16 – 14th Avenue, Seattle, WA 98128	Ellen Layman	Assistant	9
S4612	B002	City Center Plaza, Seattle, WA 98122	Dave Sinclair	Assistant	14
S4612	B004	16 – 14th Avenue, Seattle, WA 98128	Dave Sinclair	Assistant	10



Composite primary key

TempStaffAllocation

staffNo	*branchNo*	*branchAddress*	*name*	*position*	*hoursPerWeek*
S4555	B002	City Center Plaza, Seattle, WA 98122	Ellen Layman	Assistant	16
S4555	B004	16 – 14th Avenue, Seattle, WA 98128	Ellen Layman	Assistant	9
S4612	B002	City Center Plaza, Seattle, WA 98122	Dave Sinclair	Assistant	14
S4612	B004	16 – 14th Avenue, Seattle, WA 98128	Dave Sinclair	Assistant	10

Take copy of branchNo

Remove branchAddress column to new table

Take copy of staffNo

Remove position column to new table

Remove name column to new table

Branch

branchNo	*branchAddress*
B002	City Center Plaza, Seattle, WA 98122
B004	16 – 14th Avenue, Seattle, WA 98128

TempStaff

staffNo	*name*	*position*
S4555	Ellen Layman	Assistant
S4612	Dave Sinclair	Assistant

Becomes primary key

Becomes primary key

TempStaffAllocation

staffNo	*branchNo*	*hoursPerWeek*
S4555	B002	16
S4555	B004	9
S4612	B002	14
S4612	B004	10

Becomes foreign key

Composite primary key

Becomes foreign key

10. Examine the table shown below.

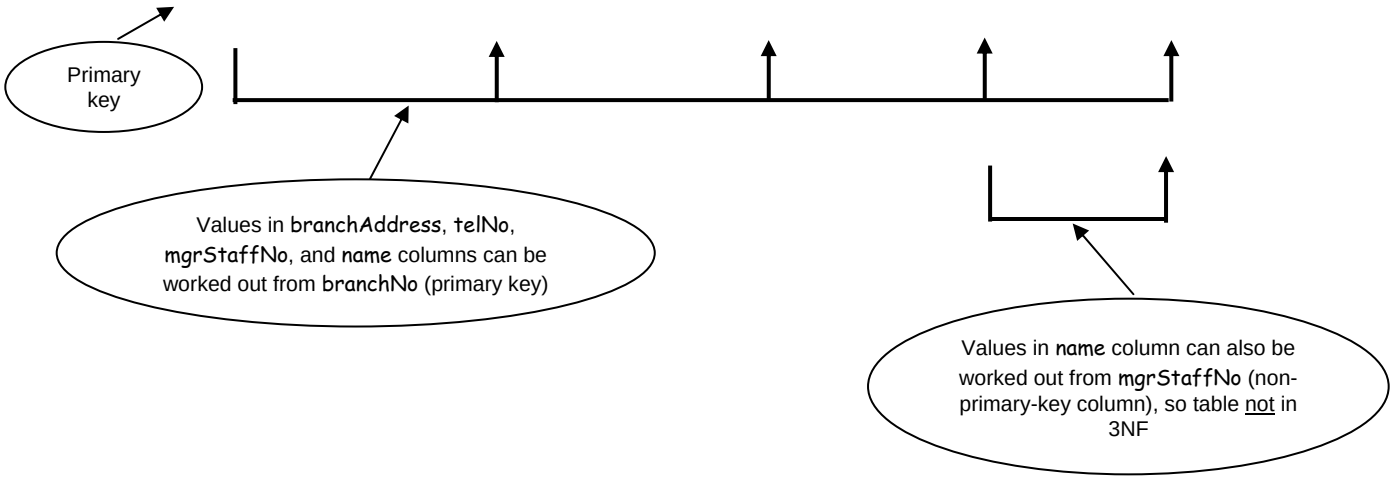
branchNo	*branchAddress*	*telNo*	*mgrStaffNo*	*name*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618	S1500	Tom Daniels
B002	City Center Plaza, Seattle, WA 98122	206-555-6756	S0010	Mary Martinez
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000	S0145	Art Peters
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131	S2250	Sally Stern

(a) Why is this table not in 3NF?
(b) Describe and illustrate the process of normalizing the data shown in this table to third normal form (3NF).
(c) Identify the primary, (alternate) and foreign keys in your 3NF relations.

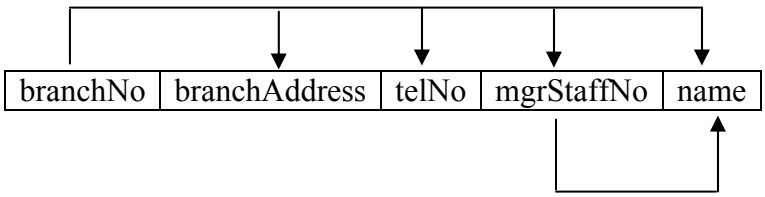
Answer:

BranchManager

branchNo	*branchAddress*	*telNo*	*mgrStaffNo*	*name*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618	S1500	Tom Daniels
B002	City Center Plaza, Seattle, WA 98122	206-555-6756	S0010	Mary Martinez
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000	S0145	Art Peters
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131	S2250	Sally Stern



Using Dependency Diagram, it can be represented as shown in the following Figure:



BranchManager

branchNo	*branchAddress*	*telNo*	*mgrStaffNo*	*name*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618	S1500	Tom Daniels
B002	City Center Plaza, Seattle, WA 98122	206-555-6756	S0010	Mary Martinez
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000	S0415	Art Peters
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131	S2250	Sally Stern

Primary key

Take copy of mgrStaffNo

Remove name column to new table

BranchManger table is renamed Branch

Branch

branchNo	*branchAddress*	*telNo*	*mgrStaffNo*
B001	8 Jefferson Way, Portland, OR 97201	503-555-3618	S1500
B002	City Center Plaza, Seattle, WA 98122	206-555-6756	S0010
B003	14 – 8th Avenue, New York, NY 10012	212-371-3000	S0415
B004	16 – 14th Avenue, Seattle, WA 98128	206-555-3131	S2250

ManagerStaff

mgrStaffNo	*name*
S1500	Tom Daniels
S0010	Mary Martinez
S0415	Art Peters
S2250	Sally Stern

Primary key

Becomes foreign key

Primary key

More Questions and Answers

1. What is normalization?

   Normalization is the process for assigning attributes to entities. Properly executed, the normalization process eliminates uncontrolled data redundancies, thus eliminating the data anomalies and the data integrity problems that are produced by such redundancies. Normalization does not eliminate data redundancy; instead, it produces the carefully controlled redundancy that lets us properly link database tables.

2. When is a table in 1NF?

A table is in 1NF when all the key attributes are defined (no repeating groups in the table) and when all remaining attributes are dependent on the primary key. However, a table in 1NF still may contain partial dependencies, i.e., dependencies based on only part of the primary key.

3. When is a table in 2NF?

A table is in 2NF when it is in 1NF and it includes no partial dependencies. However, a table in 2NF may still have transitive dependencies, i.e., dependencies based on attributes that are not part of the primary key.

4. When is a table in 3NF?

A table is in 3NF when it is in 2NF and it contains no transitive dependencies.

5. When is a table in BCNF?

A table is in Boyce-Codd Normal Form (BCNF) when it is in 3NF and every determinant in the table is a candidate key. For example, if the table is in 3NF and it contains a nonprime attribute that determines a prime attribute, the BCNF requirements are not met. This description clearly yields the following conclusions:

- If a table is in 3NF and it contains only one candidate key, 3NF and BCNF are equivalent.
- BCNF can be violated only if the table contains more than one candidate key. Putting it another way, there is no way that the BCNF requirement can be violated if there is only one candidate key.

6. Given the dependency diagram shown in the following Figure, answer the following questions.

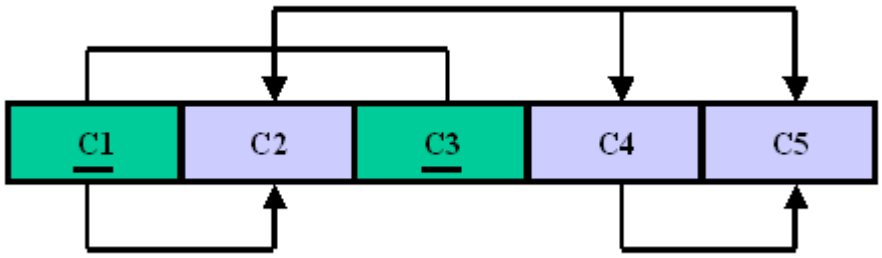


Figure: The Dependency Diagram

a. Identify and discuss each of the indicated dependencies.

- C1 → C2 represents a partial dependency, because C2 depends only on C1, rather than on the entire primary key composed of C1 and C3.
- C4 → C5 represents a transitive dependency, because C5 depends on an attribute (C4) that is not part of a primary key.
- C1, C3 → C2, C4, C5 represents a functional dependency, because C2, C4, and C5 depend on the primary key composed of C1 and C3.